

ISEL - PIV

Semestre 2024/25

Trabalho 2.A - Estimação e classificação de movimento

Trabalho realizado por:

- Miguel Alcobia, número 50746
- Tomás Salvador, número 50766

Grupo 8

Turma 51D

Docente: Pedro Mendes Jorge

Índice

1. [Introdução](#)
2. [Desenvolvimento - Trabalho Prático 2](#)
 - A. [Imports](#)
 - B. [Definição de variáveis](#)
 - C. [Processamento da imagem e criação da máscara](#)
 - D. [Funções para o jogo](#)
 - E. [Função main\(\)](#)
3. [Conclusão](#)

Introdução

Este trabalho apresenta o desenvolvimento de um algoritmo que estima e classifica movimentos, através da sua captura em vídeo. O grupo optou por integrar o algoritmo desenvolvido num jogo, onde o objetivo é o utilizador controlar um quadrado através dos seus movimentos e, com isso, conseguir "comer" os outros quadrados dispostos na grelha do jogo.

Para a classificação de movimentos, utilizou-se o conhecimento adquirido nas aulas teóricas e parte de um dos códigos apresentados nas aulas práticas para uma melhor deteção da pele. Para o desenho do contorno da mão, além da deteção da pele, também foram utilizados conceitos da matéria abordada no TP1 como a binarização e a aplicação de operadores morfológicos em máscaras binárias.

Desenvolvimento do Trabalho Prático 2

Imports

```
In [67]: import random
import cv2
import numpy as np
```

Definição de variáveis

Neste primeiro bloco de código, definiram-se algumas das variáveis que serão utilizadas ao longo do trabalho. `WIDTH` e `HEIGHT` definem as dimensões da janela onde correrá o jogo.

O tamanho do lado do quadrados foi definido como `50` na variável `square_size` e, em seguida, definiu-se a posição inicial do quadrado na janela.

São definidas as cores a atribuir aos quadrados e ao fundo do jogo. Em seguida foram criados os quadrados, com um tamanho variável desde de 50 a 100 de lado, que o utilizador terá de "comer". As informações da posição e do tamanho foram guardadas num dicionário para serem utilizadas mais tarde.

Durante as aulas práticas, na resolução dos exercícios propostos, foi apresentado um código que utilizava um intervalo de valores YCrCb para detetar regiões da pele. Para este trabalho o grupo decidiu utilizar esse mesmo intervalo de valores YCrCb. O sistema YCrCb é um bom sistema para atingir o objetivo do trabalho, porque separa o brilho (Y) da cor, o que permite a deteção da pele em diferentes condições de luminosidade.

No YCrCb os valores representam os seguintes conceitos:

- **Y**: Luminância (brilho)
- **Cr**: Crominância para vermelho
- **Cb**: Crominância para azul

Por fim, `movement_history` é um array que sevirá para guardar os movimentos detados. A partir deste histórico iremos, mais à frente, definir um **movimento dominante** para também ajudar a evitar ruído na leitura do movimento e dar mais robustez á identificação do movimento.

```
In [68]: # Tamanho da janela do jogo
WIDTH, HEIGHT = 800, 600

# Tamanho do quadrado e a posição inicial
square_size = 50
square_pos = [WIDTH // 2 - square_size // 2, HEIGHT // 2 - square_size // 2]

# Cor dos quadrados e do fundo (BGR)
RED = (0, 0, 255)
GREEN = (0, 255, 0)
WHITE = (255, 255, 255)

# número de quadrados verdes
```

```

num_green_squares = 10
# posicionar e dimensionar
green_squares = []
for _ in range(num_green_squares):
    size = random.randint(square_size, square_size * 2)
    pos_x = random.randint(0, WIDTH - size)
    pos_y = random.randint(0, HEIGHT - size)
    green_squares.append({"pos": [pos_x, pos_y], "size": size})

# intervalo em YCrCb para a pele
min_YCrCb = np.array([0, 140, 97], np.uint8)
max_YCrCb = np.array([255, 173, 127], np.uint8)

# Histórico de movimentos
movement_history = []

```

Processamento da imagem e criação da máscara

Na `main()` começa-se por transformar o sistema de cores do frame lido da câmara/vídeo de BGR para YCrCb. Em seguida, com a ajuda da função `cv2.inRange` define-se a máscara da pele (`skin_mask`), baseada no intervalo de valores YCrCb definido anteriormente. Cria-se um kernel em forma de elipse com um tamanho de 15x15 e depois aplicam-se operadores morfológicos como `CLOSE` (Preenche buracos dentro das áreas brancas) e `OPEN` (Remove pequenos ruídos ao fazer erosão e depois dilatação).

Por fim, faz-se uma dilatação para aumentar as áreas brancas para garantir que não existam buracos ou desconexões no interior da máscara, como a ponta de um dedo não estar unida ao dedo em si. Finalizou-se com a função `cv2.threshold()` converte a máscara processada numa imagem binário, onde todos os valores acima de 127 se tornam 255.

```

In [ ]: def process_mask(mask):
        """Faz a máscara que destaca a região da pele

        Args:
            mask: máscara binária com a região da pele destacada

        Returns:
            binary_mask: máscara binária após a aplicação dos operadores morfo

        """
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (15, 15), (-1, -1))
        processed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
        processed = cv2.morphologyEx(processed, cv2.MORPH_OPEN, kernel)
        dilated = cv2.dilate(processed, kernel, iterations=2)
        _, binary_mask = cv2.threshold(dilated, 127, 255, cv2.THRESH_BINARY)
        return binary_mask

```

Para identificar os contornos da máscara da pele no vídeo, de forma a localizar a posição e o formato da mão foi desenvolvida a função `draw_convex_hull()`. Esta função usa o método Convex Hull, que desenha a forma geométrica que envolve a mão, eliminando concavidades de forma a garantir que o formato seja o mais simples

possível. Desta forma, será possível acompanhar a posição da mão ao fazer os movimentos, e posteriormente classificá-los.

Esta função utiliza o método `findContours()` para detetar os contornos presentes na máscara, mantendo apenas os contornos externos (`cv2.RETR_EXTERNAL`) para evitar ruídos desnecessários, caso existam.

Após detetar os contornos, filtram-se aqueles cuja área seja maior do que o valor definido, neste caso optou-se por 50, pois após alguns testes pareceu ser um valor aceitável. Desta forma garantiu-se que apenas fossem considerados contornos mais significativos, como é o caso da mão, ignorando pequenos ruídos que também poderiam ser detetados e afetar a identificação dos movimentos. Para os contornos que estão dentro do limite desejado, utilizou-se o `cv2.convexHull()`, para criar uma linha fechada que envolve completamente o contorno, mesmo em áreas mais complexas como no caso dos dedos. É essa a linha que aparece representada a verde em tempo real na câmara.

Para se poder detetar melhor a posição da mão, foi calculado o centróide de cada contorno significativo. Através dos `cv2.moments()`, são obtidos os momentos geométricos do contorno, e calcula-se o centróide, que aparece representado a vermelho na câmara.

Assumiu-se que o maior contorno encontrado seria o contorno da mão, e por isso retorna-se a sua área e a posição do centroide associado a esse contorno para mais tarde usar estes valores na deteção de movimento. O área será o valor base para verificar a existência de zoom e a posição do centróide servirá para ver em que direção a mão se está a mover.

```
In [70]: def draw_convex_hull(frame, mask, min_area=50):
        """Desenha o Convex Hull

        Args:
            frame: frame lido pela câmara
            mask: máscara binária que destaca a pele
            min_area: Área mínima para desenhar contornos. Default a 50.

        Returns:
            frame, largest_centroid, largest_area: frame lido pela câmara, ce
        """
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        largest_centroid = None
        largest_area = 0 # Mantem a maior área do contorno

        # Desenha o convex Hull apenas para contornos que tenham uma área mai
        for contour in contours:
            area = cv2.contourArea(contour)
            if area >= min_area:
                hull = cv2.convexHull(contour)
                cv2.drawContours(frame, [hull], 0, (0, 255, 0), 2) # Verde p

                moments = cv2.moments(contour)
                if moments["m00"] != 0:
                    cx = int(moments["m10"] / moments["m00"])
```

```

        cy = int(moments["m01"] / moments["m00"])
        largest_centroid = (cx, cy)
        cv2.circle(frame, (cx, cy), 5, (0, 0, 255), -1) # Vermelha

        # Armazenar a maior área do contorno
        if area > largest_area:
            largest_area = area

    return frame, largest_centroid, largest_area

```

Para identificar a direção do movimento da mão entre frames consecutivos, desenvolveu-se a função `detect_direction()`. Começa-se por comparar os centroides da mão em dois frames seguidos, e calcula-se o deslocamento horizontal e vertical entre eles. Desta forma é possível identificar as direções dos movimentos efetuados pela mão.

A função `detect_direction()` determina a direção do movimento da mão entre dois frames consecutivos, com base nas posições dos centroides calculadas anteriormente. Inicialmente, calculam-se os deslocamentos horizontal (`delta_x`) e vertical (`delta_y`) entre as posições dos centroides. Para evitar detecções incorretas causadas por ruídos foi utilizado um limite mínimo (`movement_threshold`), que garante que apenas movimentos maiores do que 10px sejam aceites. Este valor foi escolhido após a realização de alguns testes, e pareceu ser um valor adequado.

Para determinar a direção dos movimentos, são feitas comparações entre os deslocamentos vertical e horizontal. Dessas comparações resulta uma string com o movimento correspondente ou um string vazia, que indica a ausência de movimento.

```

In [ ]: def detect_direction(prev_centroid, current_centroid, movement_threshold=
        """Deteta a direção

        Deteta a direção com base na diferença entre a posição dos centroides

        Args:
            prev_centroid: centroide anterior
            current_centroid: centroide atual
            movement_threshold: limiar com a quantidade de pixels que o movim

        Returns:
            movimento detetado
        """
        if prev_centroid and current_centroid:
            prev_cx, prev_cy = prev_centroid
            curr_cx, curr_cy = current_centroid

            # Calcula o deslocamento horizontal e vertical
            delta_x = curr_cx - prev_cx
            delta_y = curr_cy - prev_cy

            # Verifica que o movimento é significativo com base no threshold
            if abs(delta_x) > abs(delta_y) and abs(delta_x) > movement_threshol
                if delta_x > 0:
                    return "Direita"
                else:
                    return "Esquerda"

```

```

        elif abs(delta_y) > abs(delta_x) and abs(delta_y) > movement_thre
            if delta_y > 0:
                return "Baixo"
            else:
                return "Cima"
    return ""

```

Para detetar o zoom in ou zoom out, decidiu-se que a melhor abordagem seria a variação da área do contorno da mão entre dois momentos consecutivos.

Calcula-se a diferença entre as áreas e divide-se o resultado pela área mais antiga, ficando assim com a diferença relativa das áreas. Este valor é comparado com um valor (`zoom_threshold`) e, caso este seja menor do que o resultado da diferença anterior, temos um Zoom In, caso contrário temos um Zoom Out.

```

In [ ]: def detect_zoom_in_out(prev_area, curr_area, zoom_threshold=0.1):
        """Deteta Zoom

        Deteta Zoom com base na diferença entre a área do contorno da mão

        Args:
            prev_area: área anterior
            curr_area: área atual
            zoom_threshold: limiar para o zoom. Defaults a 0.1.

        Returns:
            tipo de zoom
        """
        # Calcula a razão entre as áreas e define se houve Zoom in ou Zoom ou
        if prev_area and curr_area:
            area_change = (curr_area - prev_area) / prev_area
            if area_change > zoom_threshold:
                return "Zoom In"
            elif area_change < -zoom_threshold:
                return "Zoom Out"
        return ""

```

Durante os testes que foram realizados ao longo do trabalho, verificou-se que quando se queria executar um movimento para a direita eram executados alguns movimentos "secundários" para baixo e para cima, o que no contexto escolhido não era o ideal. Com o objetivo de remover esses movimentos que criam ruído foi criada a função abaixo para identificar o movimento predominante num conjunto de movimentos registrados ao longo do tempo. Assim já é possível filtrar ruídos e garantir que pequenos desvios ou movimentos inconsistentes da mão não afetem a precisão do movimento "principal" do quadrado, mantendo a direção predominante dos gestos realizados pela mão.

Portanto, a função `get_dominant_movement()` começa por verificar se a lista de movimentos não está vazia e, caso não esteja, utiliza a função `max()` e a função `count()` para encontrar o elemento da lista que aparece com maior frequência. O movimento mais comum é então retornado como o movimento dominante.

```

In [ ]: # Determina o movimento dominante apenas quando há novos movimentos
def get_dominant_movement(history):

```

```
"""Descobre o movimento dominante
```

Com o objetivo de evitar a interferência de pequenos ruídos, procura-se no histórico de movimentos qual foi o que se repetiu mais vezes. Considera-se esse movimento como o movimento dominante e é essa a direção/ação que o quadrado deve executar.

Args:

history: histórico com os movimentos capturados

Returns:

movimento dominante: movimento que mais se repete no histórico

```
"""
```

```
if not history:
```

```
    return ""
```

```
return max(history, key=history.count)
```

Funções para o jogo

A função abaixo verifica se existe colisão entre dois quadrados, com base nas suas posições e tamanhos.

```
In [74]: def is_collision(square1, square2):
    """Deteta se existe colisão nos quadrados do jogo

    Args:
        square1: quadrado 1
        square2: quadrado 2

    Returns:
        `False` se não houver colisão, `True` caso contrário
    """
    x1, y1, size1 = square1["pos"][0], square1["pos"][1], square1["size"]
    x2, y2, size2 = square2["pos"][0], square2["pos"][1], square2["size"]
    return not (x1 + size1 < x2 or x1 > x2 + size2 or y1 + size1 < y2 or
```

Para executar as ações baseadas no movimento dominante detetado foi criada a função `execute_dominant_movement`. Consoante o movimento, o quadrado vermelho na interface do jogo, desloca-se para os lados, para cima ou para baixo, ou faz de zoom in e zoom out. Após isto os movimentos da mão já vão ser refletidos no jogo.

```
In [ ]: def execute_dominant_movement(dominant_movement, square_pos, square_size,
    """
    Executa a ação com base no movimento dominante.

    Args:
        dominant_movement: Movimento dominante detetado.
        square_pos: Posição atual do quadrado [x, y].
        square_size: Tamanho atual do quadrado.
        zoom_index: Índice atual do nível de zoom.
        zoom_levels: Lista com os níveis possíveis de zoom.

    Returns:
        square_size, zoom_index: Tamanho do quadrado, e índice de zoom atual.
    """
    move_square = 50 # Quantidade de deslocamento do quadrado.
```

```

if dominant_movement == "Direita":
    square_pos[0] = min(WIDTH - square_size, square_pos[0] + move_square)
elif dominant_movement == "Esquerda":
    square_pos[0] = max(0, square_pos[0] - move_square)
elif dominant_movement == "Baixo":
    square_pos[1] = min(HEIGHT - square_size, square_pos[1] + move_square)
elif dominant_movement == "Cima":
    square_pos[1] = max(0, square_pos[1] - move_square)
elif dominant_movement == "Zoom In":
    if zoom_index < len(zoom_levels) - 1:
        zoom_index += 1
        square_size = zoom_levels[zoom_index]
elif dominant_movement == "Zoom Out":
    if zoom_index > 0:
        zoom_index -= 1
        square_size = zoom_levels[zoom_index]

return square_size, zoom_index

```

A função jogo cria e atualiza o jogo na sua janela. A janela do jogo possui:

- Quadrados verdes: Alvos a "comer".
- Quadrado vermelho: O quadrado controlado pelo jogador.
- Grid: Uma grelha para ajudar a ver o deslocamento do quadrado após o movimento.
- Texto do movimento: Mostra o movimento realizado pelo jogador.

Além disso, a função verifica se o quadrado vermelho colide com os quadrados verdes e, em caso de colisão, remove os quadrados verdes menores ou iguais em tamanho.

```

In [ ]: def jogo():
    """Função que desenha a janela onde está o jogo.

    Returns:
        janela do jogo
    """

    # Janela para o jogo
    game_window = np.full((HEIGHT, WIDTH, 3), WHITE, dtype=np.uint8)

    # Desenhar os quadrados verdes
    for green_square in green_squares:
        pos_x, pos_y = green_square["pos"]
        size = green_square["size"]
        cv2.rectangle(game_window, (pos_x, pos_y), (pos_x + size, pos_y + size), GREEN, 1)

    # Adicionar a grid na janela
    grid_spacing = 50
    for x in range(0, WIDTH, grid_spacing):
        cv2.line(game_window, (x, 0), (x, HEIGHT), (200, 200, 200), 1)
    for y in range(0, HEIGHT, grid_spacing):
        cv2.line(game_window, (0, y), (WIDTH, y), (200, 200, 200), 1)

    # Desenhar o quadrado vermelho
    cv2.rectangle(game_window, (square_pos[0], square_pos[1]), (square_pos[0] + square_size, square_pos[1] + square_size), RED, 1)

```

```

        (square_pos[0] + square_size, square_pos[1] + square_size,
         RED, -1)

    # Verifica se houve colisão com os quadrados verdes
    for green_square in green_squares[:]:
        if is_collision({"pos": square_pos, "size": square_size}, green_square):
            if green_square["size"] <= square_size:
                green_squares.remove(green_square)

    # Exibir o texto do movimento no canto superior esquerdo
    text_position = (10, 30)
    movement_text = dominant_movement if pause_counter == 0 else ""
    cv2.putText(game_window, movement_text, text_position,
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2, cv2.LINE_AA)

    return game_window

```

Função main()

A função `main()` chama todas as funções criadas anteriormente e dá vida ao jogo, permitindo capturar os movimentos da mão, identificá-los e traduzi-los em ações no jogo.

Começa-se por inicializar as variáveis, sendo as mais importantes a variável `pause_counter`, que é um contador que controla o número de frames já processados para parar a leitura temporariamente ao atingir 20 frames lidos. Esta pausa é realizada para evitar ruído na detecção dos movimentos.

Os `frames_detect` indicam o número de frames necessário para validar um movimento dominante. `frames_pause` indica o número de frames em que a captura é interrompida para evitar ruído. `zoom_levels` define os tamanhos que o quadrado do utilizador pode assumir e `zoom_index`, indica o nível de zoom inicial.

A captura pode ser feita através da câmara do PC com `cv2.VideoCapture(0)` ou através de vídeo já gravado. Para tal, basta mudar o `0` para o path do ficheiro do vídeo (`video_path`).

Depois começa-se o processamento das frames lidas, criando a máscara binária que destaca a região da pele e desenhando o contorno com o Convex Hull. O frame que é lido também é invertido, para que os movimentos não fiquem espelhados e sejam mais intuitivos.

Em seguida, começa-se por verificar a existência de zoom e caso não exista, passa-se à detecção de movimentos (esquerda, baixo, cima, direita):

- Caso o `pause_counter` esteja a zero, ativa-se a captura de movimento. Caso contrário, o seu valor é decrementado.
- Detecta-se o movimento (Esquerda, Direita, Cima, Baixo) caso não haja zoom e o movimento é colocado no histórico.
- Verifica-se o tamanho do histórico para possuir que este tenha apenas o número de frames necessários para a detecção de movimento.

- Caso o tamanho do histórico seja igual ao número de frames necessários, obtém-se o movimento dominante e depois executa-se a função

`execute_dominant_movement()` para que o comando do movimento seja executado pelo quadrado.

```
In [ ]: def main():
        """
        Função Main
        """

        # variáveis globais
        global min_YCrCb, max_YCrCb, square_size, movement_history, dominant_

        # Inicialização das variáveis
        prev_centroid = None # Centroide da mão
        prev_area = None # Área da mão
        dominant_movement = "" # String para o movimento dominante

        frames_to_detect = 5 # Número de frames para validar o movimento dom
        frames_pause = 20 # Número de frames de pausa após capturar o movime
        pause_counter = 0 # Contador para a pausa da captura de movimento
        zoom_levels = [square_size, square_size * 2, square_size * 4] # Tama
        zoom_index = 0 # Índice atual no zoom_levels (começa com o tamanho o

        # Captura de video

        """VIDEO GUARADO NO PC"""
        #video_path = "D:\\ISEL\\5ºSemestre\\PIV\\TP2\\WIN_20241202_08_58_39_
        #cap = cv2.VideoCapture(video_path)

        """VIDEO COM CÂMARA"""
        cap = cv2.VideoCapture(0)

        while True:
            # leitura do frame da captura
            ret, frame = cap.read()
            if not ret:
                break

            frame = cv2.flip(frame, 1) # Flip horizontal para não espelhar o
            ycrb = cv2.cvtColor(frame, cv2.COLOR_BGR2YCrCb)
            skin_mask = cv2.inRange(ycrb, min_YCrCb, max_YCrCb)
            skin_mask = process_mask(skin_mask)

            frame_with_hull, current_centroid, current_area = draw_convex_hul

            # Detectar zoom in/out
            zoom_action = detect_zoom_in_out(prev_area, current_area)

            if pause_counter == 0:
                # Capturar movimento ou zoom
                movement_direction = detect_direction(prev_centroid, current_
                # se não for zoom deteta um movimento normal
                if zoom_action == "Zoom In":
                    movement_history.append("Zoom In")
                elif zoom_action == "Zoom Out":
                    movement_history.append("Zoom Out")
                elif movement_direction:
```

```

        movement_history.append(movement_direction)

        # Verificar o movimento dominante
        if len(movement_history) == frames_to_detect:
            dominant_movement = get_dominant_movement(movement_history)

            # Executar ações com base no movimento dominante
            square_size, zoom_index = execute_dominant_movement(dominant_movement)

            # Resetar o histórico e iniciar a pausa
            movement_history = []
            pause_counter = frames_pause # 20 frames

        else:
            # Reduzir o contador da pausa
            pause_counter -= 1

        # Atualizar centroides e áreas para o próximo frame
        prev_centroid = current_centroid
        prev_area = current_area

        game_window = jogo()

        # Mostrar as janelas
        cv2.imshow("Câmara", frame_with_hull) # Câmara
        cv2.imshow("Skin mask", skin_mask) # máscara binária a detetar a mão
        cv2.imshow("Jogo", game_window) # janela do jogo

        # Sair ao pressionar 'q'
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

Conclusão

Este trabalho ajudou de forma geral, a perceber como funcionam os algoritmos para capturar movimentos que são bastantes utilizados em jogos para a realização de animações e até como forma de interação do jogador com a consola, como é o caso do Nintendo Wii.

A parte que se revelou mais complicada foi separar a mão na máscara binária e ignorar pequenos ruídos que apareciam na parede, quando esta tinha uma cor que se assemelhava à da pele. Se estivessemos com uma camisola cuja a cor também fosse parecida à da pele, o contorno da mão crescia para apanhar o braço. Esse problema foi ultrapassado através de alguns testes com valores diferentes.

A realização do projeto acabou por ser divertida, também pelo facto de ter sido criado um jogo, mesmo que simples, pois permitiu dar mais vida ao algoritmo criado ao colocá-lo num contexto prático e plausível se pensarmos no dia a dia.